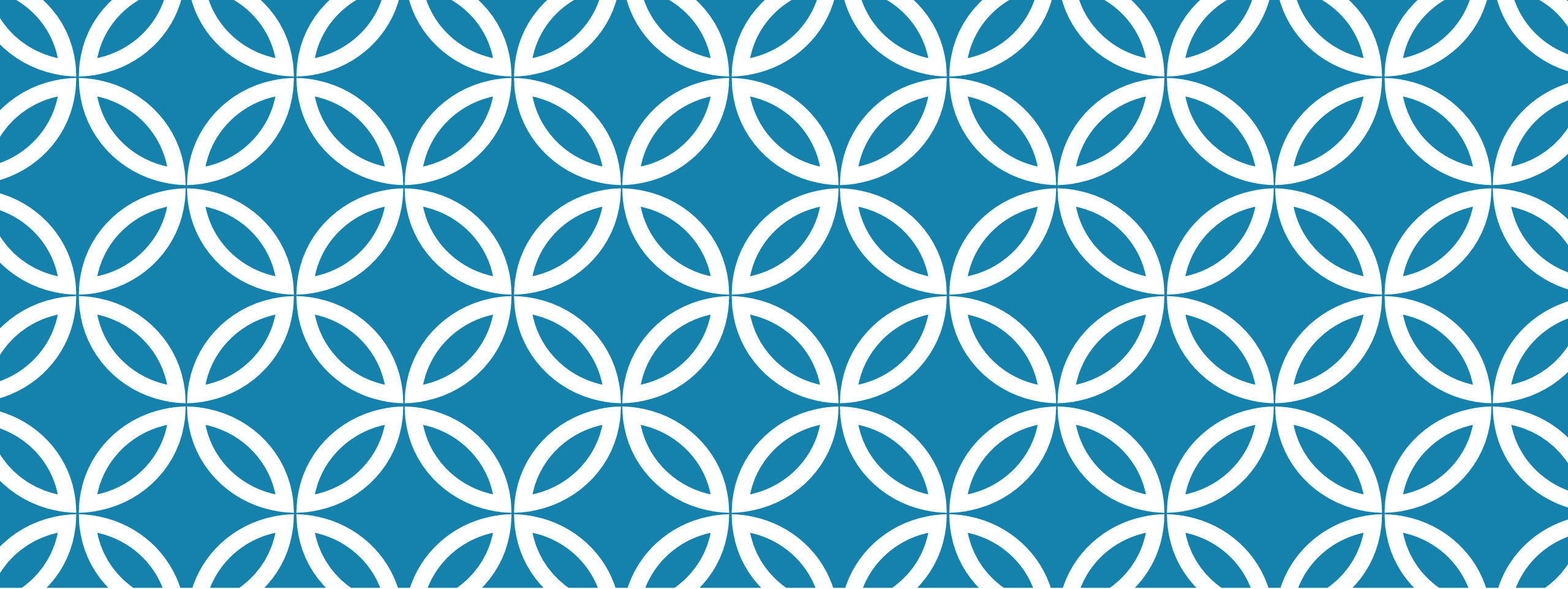


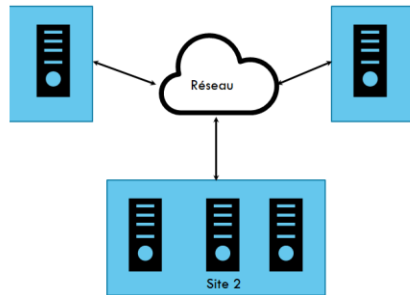


TECHNOLOGIES & SERVICES WEB

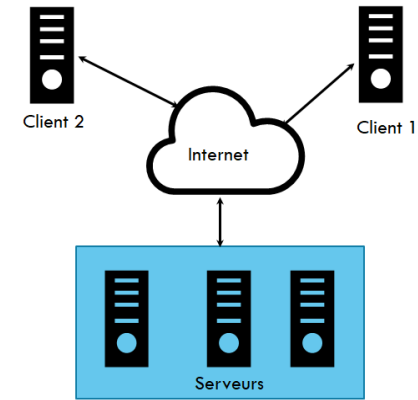
XML, SOAP, UDDI, CXF et les
services web

M. Flavien SOMDA

ARCHITECTURES RÉPARTIES



De nos jours, les architectures au sein des entreprises sont pour la plupart des architectures réparties



Les architectures client-serveur rendent directement accessibles aux consommateurs les informations détenues par les fournisseurs

ARCHITECTURES RÉPARTIES

Par exemple, une banque de dépôt offre à ses clients, la possibilité de consulter leurs comptes par divers moyens:

➤ Mobile

➤ Internet

Dans cet exemple, on dispose de plusieurs applications qui échangent via un réseau.

- Une application de mise en forme pour l'affichage Web (Frontal web)
- Une application de mise en forme pour l'affichage Mobile (Frontal mobile)
- Une application de traitement permettant de fournir aux applications frontales les données à afficher selon les demandes de l'utilisateur.

ARCHITECTURES RÉPARTIES

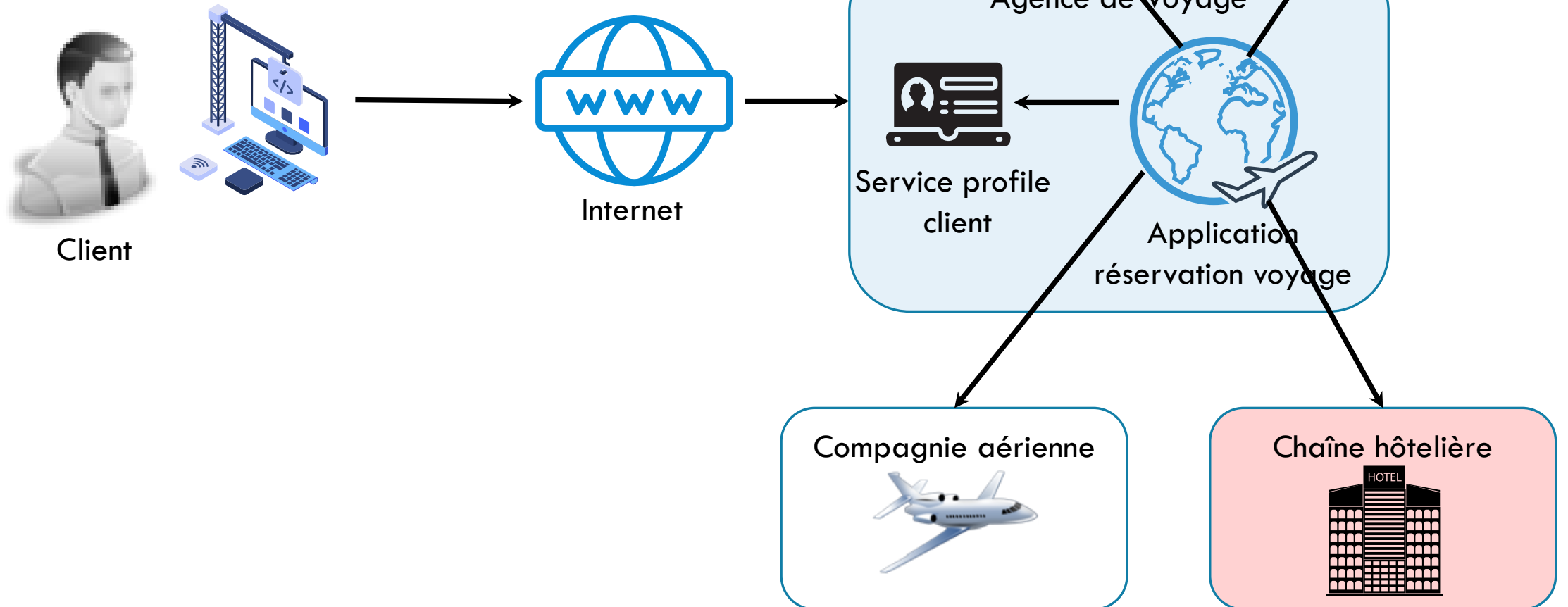
De même la banque peut mettre à disposition de ses partenaires certaines données dont elle dispose pour des besoins commerciaux. Ces partenaires peuvent être des compagnies d'assurance ou des sociétés financières dont la banque commercialise les produits.

- Les Systèmes d'information de ces partenaires doivent pouvoir échanger avec celui de la banque et inversement alors qu'ils n'appartiennent pas au même réseau.
- L'architecture distribuée doit donc être ouverte et flexible afin de faciliter ces échanges B2B
- Les services mis à disposition des partenaires doivent pouvoir être fournis à partir de services déjà existants moyennant éventuellement une orchestration de ces derniers.

ARCHITECTURES RÉPARTIES

- Un autre exemple. Considérons le système d'information d'une agence de voyage. Les produits commercialisés sont donc des voyages; Mais chaque voyage est une agrégation d'autres sous-produits:
 - Billets de transport
 - Nuits d'hôtel
 - Voiture de location
 - Billets d'entrée sur des sites touristiques, etc.
- Pour élaborer un produit « voyage », le système d'information de l'agence doit donc interroger les systèmes d'information d'autres fournisseurs:
 - Compagnies aériennes et ferroviaires
 - Chaînes hôtelières
 - Sociétés de location de véhicules
 - Sociétés de commercialisation de billets pour les sites touristiques

ARCHITECTURES RÉPARTIES



ARCHITECTURES RÉPARTIES

- Sur le schéma précédent, l'application sollicitée par le client pour son voyage orchestre un ensemble d'applications partenaires réparties(sur des sites et des réseaux différents) pour composer le voyage
 - On dit qu'elle agrège ces applications
- L'application sollicitée consulte également un service de profile afin d'adapter le dialogue avec les applications partenaires en fonction de l'utilisateur
 - On parle de transformation
- Les composants précédemment cités, qui agissent de manière automatique et opaque en exposant leurs fonctionnalités sur un réseau s'appellent des « services web »
- Une application répartie est donc un assemblage de services web pouvant être internes (comme le service de profile client) ou externes (comme le service de réservation de billets d'avion).

ARCHITECTURES RÉPARTIES

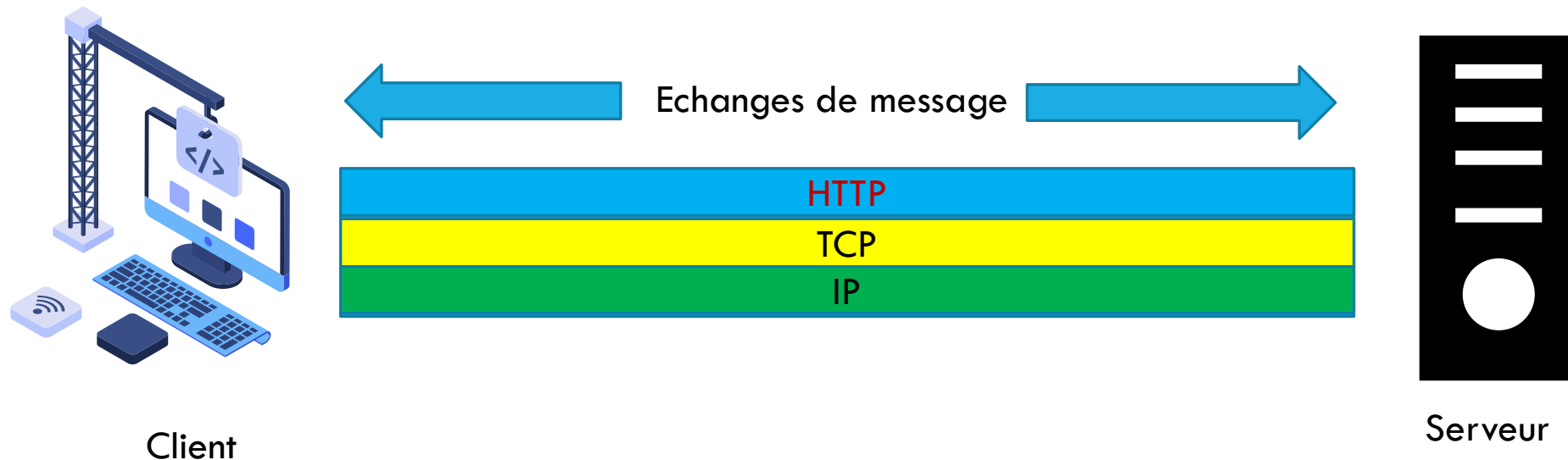
- Comme dans l'exemple de la compagnie de voyage, une application communicante peut consister en un assemblage mixte de services internes et externes.
- Les deux extrémités possibles de cet assemblage sont:
 - Les applications d'entreprise entièrement constituées de services internes: on parle alors d'intégration d'applications d'entreprise EAI (Enterprise Application Integration)
 - Les applications entièrement constituées de services externes: on parle alors de portail d'entreprises.

ARCHITECTURES RÉPARTIES

- Il est important de noter que les web services ne sont pas la seule manière de mettre en œuvre une architecture distribuée.
- Avant les services web, les architectures réparties ont été réalisées en recourant à:
 - Des protocoles de communication (Corba, DCOM, RMI)
 - Des langages d'implémentation (Java, Smaltalk, C++, ...)
 - Des interfaces d'interaction et de présentation
- Néanmoins, l'avènement du web a imposé le HTTP comme protocole d'échange universel, ce dernier étant caractérisé par sa légèreté par rapport aux anciens protocoles comme Corba ou RMI et nécessitant un simple navigateur web pour matérialiser les échanges.
- Les services web sont simplement la mise en œuvre des architectures distribuées par le biais des technologies du web.

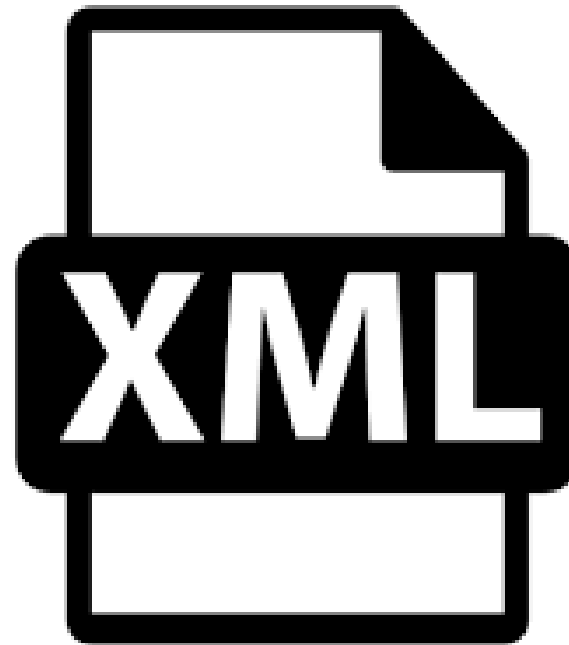
PRINCIPES

- Les services web reposent sur le transport d'une demande de service entre un client et un serveur.
- Ce transport est assuré par la suite de protocoles « internet/web » que sont TCP/IP et HTTP.



PRINCIPES

- HTTP transporte exclusivement du texte
- Les échanges entre le client et le serveur se font donc au format texte
- Dans le cas des services web, le message textuel est au format XML
- L'apport majeur des services web est le passage du web client, celui des échanges entre un client léger (en général un navigateur) et un serveur au web machines, c'est-à-dire celui des échanges directs entre applications.
- Les services web ont également induit une évolution majeure dans les technologies concernant les architectures distribuées: passage des technologies RPC, Corba, DCOM aux technologies HTTP et XML



LE LANGUAGE XML

LIMITES DU LANGAGES HTML

- Le HTML est le premier langage auquel on pense dès qu'on parle du web
- Pourtant dans l'implémentation des services web, le HTML n'est pas le langage utilisé pour les échanges textuels de messages
- Cela s'explique par différentes raisons:
 - Le HTML est très bien adapté à la construction des pages web, il est bien compris par les navigateurs et permet de mettre facilement en forme les pages web. Cependant, le HTML souffre de multiples limites
 - Il est composé d'un ensemble de balises prédéfinies et ne permet pas de créer ses propres balises
 - Il propose une gestion simpliste des liens qui ne peuvent pointer que vers d'autres pages ou parties de pages
 - Et surtout le HTML ne propose aucune sémantique métier dans la définition de ses contenus. On ne peut pas décrire formellement les éléments et liens d'un domaine métier particulier.

LIMITES DU LANGAGES HTML

<H2>Bibliographie XML</H2>

 Jean-Christophe Bernadac et François Knab, <I>Construire une application XML</I>, Paris, Eyrolles, 1999

 Alain Michard, <I>XML, Langage et Applications</I>, Paris, Eyrolles, 1998

 William J. Pardi, <I>XML en Action</I>, Paris, Microsoft Press, 1999, adapté de l'anglais par James Guerin

- Le balisage de ce texte permet au navigateur de reconnaître et mettre en forme le titre de niveau 2 ainsi que la liste non ordonnée avec ses différents éléments
- Par contre, il ne permet pas d'exprimer le fait que Jean-Christophe Bernadac et François Knab sont les **auteurs du livre** « Construire une application XML » publié au cours de **l'année** 1999 dans les **éditions** Eyrolles **situés à** Paris.
- Il est incapable d'exprimer les concepts d'auteur, de livre, d'éditeur propres au domaine des livres ou des bibliothèques

OBJECTIFS DU XML

- XML signifie (eXtensible Markup Language) et se définit comme un langage de description et d'échanges de données structurées
- Le but principal du XML est d'étendre les fonctionnalités du HTML en lui adjoignant une dimension sémantique notamment afin faciliter les échanges d'informations entre applications
- Les apports du XML sont les suivants:
 - La définition libre de nouvelles balises et de nouveaux attributs
 - Le support de structures complexes de documents
 - La vérification de la structure d'un document par rapport à une grammaire type défini dans un document appelé DTD (Document Type Descriptor)
 - La séparation entre le contenu (document XML) et la présentation (feuille de style CSS ou XSL)

EXEMPLE DE DEFINITION DTD D'UN DOCUMENT XML

La DTD suivante déclare un élément list de contenu pur. Il contient une suite non vide d'éléments item contenant chacun du texte.

`<!-- Fichier "standalone.dtd" -- >` *cette ligne représente un commentaire*

`<!ELEMENT liste (item)+ >`

`<!ELEMENT item (#PCDATA) >`

RÈGLES DE DÉFINITION DES ÉLÉMENTS DTD

- **<!ELEMENT elem(elem1, elem2, elem3) >**
 - L'élément elem doit contenir un élément elem1, un élément elem2 puis un élément elem3 dans cet ordre.
- **<!ELEMENT elem(elem1 | elem2 | elem3) >**
 - L'élément elem doit contenir un seul des éléments elem1, elem2 ou elem3.
- **<!ELEMENT elem(elem1, elem2?, elem3) >**
 - L'élément elem doit contenir un élément elem1, un ou zéro élément elem2 puis un élément elem3 dans cet ordre.
- **<!ELEMENT elem(elem1, elem2*, elem3) >**
 - L'élément elem doit contenir un élément elem1, une suite éventuellement vide d'éléments elem2 et un élément elem3 dans cet ordre.

RÈGLES DE DÉFINITION DES ÉLÉMENTS DTD (SUITE)

- **`<!ELEMENT elem(elem1, (elem2 | elem4), elem3) >`**
 - L'élément `elem` doit contenir un élément `elem1`, un élément `elem2` ou un élément `elem4` puis un élément `elem3` dans cet ordre.
- **`<!ELEMENT elem(elem1, elem2, elem3)* >`**
 - L'élément `elem` doit contenir une suite d'éléments `elem1`, `elem2`, `elem3`, `elem1`, `elem2`, ... jusqu'à un élément `elem3`.
- **`<!ELEMENT elem(elem1*, elem2*, elem3*) >`**
 - L'élément `elem` doit contenir une suite quelconque d'éléments `elem1`, `elem2` ou `elem3`.
- **`<!ELEMENT elem(elem1 | elem2 | elem3)+ >`**
 - L'élément `elem` doit contenir une suite non vide d'éléments `elem1`, `elem2` ou `elem3`.

OBJECTIFS DU XML

- XML a été conçu pour représenter les documents suivant les cinq grands principes suivants
 - Lisibilité par l'homme et la machine
 - Définition non ambiguë du contenu d'un document
 - Définition non ambiguë de la structure d'un document
 - Séparation entre documents et relations entre documents
 - Séparation entre structure du document et présentation du document

OBJECTIFS DU XML

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<BIBLIO SUBJECT="XML">
  <BOOK ISBN="9782212090819" LANG="fr" SUBJECT="applications">
    <AUTHOR>
      <FIRSTNAME> Jean-Christophe </FIRSTNAME>
      <LASTNAME> Bernadac</LASTNAME>
    </AUTHOR>
    <AUTHOR>
      <FIRSTNAME> François </FIRSTNAME>
      <LASTNAME> Knab</LASTNAME>
    </AUTHOR>
    <TITLE> Construire une application XML </TITLE>
    <PUBLISHER>
      <NAME> Eyrolles </NAME>
      <PLACE> Paris </PLACE>
    </PUBLISHER>
    <DATEPUB> 1999 </DATEPUB>
  </BOOK>
  ...
</BIBLIO>
```

OBJECTIFS DU XML

- Dans ce document XML, on constate qu'aucune des balises ne décrit la présentation finale. Par contre, on note bien que les balises ont dorénavant un vrai sens métier ainsi qu'une hiérarchie.
- Par exemple l'élément « author » comprend le prénom avec la balise « firstname » et le nom avec la balise « lastname » de l'auteur.
- De plus, certaines informations telles que la langue d'écriture du livre ou son numéro ISBN ont pu être rajoutées comme attributs directement au sein de la balise « BOOK ».

LES DOMAINES DE NOM (NAMESPACES) XML

- Le XML permet la création de document avec des balises spécialisées pour un domaine métier et réutilisable. Cette exigence de réutilisabilité nécessite que les balises constituent des concepts d'un vocabulaire métier (banque, immobilier, librairie, etc.)
- Le vocabulaire étant propre à chaque domaine métier, il est donc courant de mettre en œuvre des DTD adaptés à chaque domaine et de les référencer pour constituer un document XML.
- Mais des conflits peuvent naître du référencement de différents DTD étant donné qu'un même concept peut être exprimé dans différents métiers.
- exemple, le concept de VENTE se retrouve aussi bien chez les librairies que dans l'immobilier. Si un document XML référence à la fois un DTD immobilier et un DTD librairie et contient un élément VENTE alors il y a conflit car on ne sait pas si on fait référence à la VENTE provenant du DTD immobilier ou à la VENTE provenant du DTD librairie.

LES DOMAINES DE NOM (NAMESPACES) XML

- Afin de résoudre les conflits, la W3C a fait adopter le principe consistant à préfixer chaque élément XML par un nom unique qui identifie le domaine auquel il fait référence.
- Le recours aux préfixes apporte de nombreux bénéfices:
 - Levée des ambiguïtés sur les éléments utilisés dans le document XML
 - Modularité des documents XML: on peut composer un document en composant avec des sources multiples

LES DOMAINES DE NOM (NAMESPACES) XML

➤ On déclare un espace de nom (namespace) à l'aide de l'attribut ***xmlns:***

➤ soit en déclarant l'espace de nom dans l'élément

■ `<ElementDuDocument xmlns="UriDuDTDalImporter">`

■ Exemple: `<VENTE xmlns="www.ujkz.bf/librairie">`

➤ Soit en associant un préfixe à l'URI pour une utilisation plus simple

■ `<VENTE xmlns:lib="www.ujkz.bf/librairie">`

...

`</lib:VENTE>`

LES SCHÉMAS XML

- Une des principales limites des DTDs, c'est qu'ils ne décrivent que la structure d'un document XML. Cela suffisait pour les premières utilisations de XML. Cependant, pour permettre l'échange de données quelconques via XML, la description par des DTDs est rapidement devenue insuffisante. ET les DTDs présentent d'autres limitations:
 - Elles ne sont pas écrites en XML et donc les technologies permettant d'exploiter les documents XML comme DOM ou SAX ne peuvent pas parser les DTDs
 - Les DTDs ne supportent pas les espaces de nom. Il est donc impossible l'import de schémas existants afin de réutiliser du code existant
 - Le typage des données est très limité (pas de type Booléen ou Entier par exemple)
- Afin de permettre la description d'une structure quelconque (arbitrairement complexe) de données et de combler les lacunes des DTDs, la W3C a proposé les XML Schema comme nouveau langage de définition des documents XML; l'ambition étant de pouvoir décrire aussi bien des modèles de bases de données relationnelles que des modèles objet des langages Objet comme Java ou n'importe quel autre modèle de données.

LES SCHÉMAS XML

➤ Les schémas XML apportent les bénéfices suivants:

➤ Le support des espaces de noms

➤ Les indicateurs d'occurrence

➤ Une grande facilité de conception de documents modulaires

➤ Le support natif de nombreux types de données: Booléen, Entier, Intervalle de temps, etc.

```
<xs:element name="customer" type="xs:string"/>
```

```
<xs:attribute name="disabled" type="xs:boolean"/>
```

```
<xs:element name="start" type="xs:date"/>
```

```
<xs:element name="price" type="xs:decimal"/>
```

➤ Possibilité de créer de nouveaux types par ajout de contrainte sur un type existant. Par exemple on peut définir un type LetterType comme une chaîne de caractères n'acceptant que des caractères de a-z et A-Z. Les caractères comme « ! » ou « 1 » par exemple ne seront pas acceptés.

```
<xs:simpleType name="LetterType">
```

```
  <xs:restriction base="xs:string">
```

```
    <xs:pattern value="[a-zA-Z]" />
```

```
  </xs:restriction>
```

```
</xs:simpleType>
```

LES SCHÉMAS XML

- Les schémas XML apportent les bénéfices suivants:
 - Possibilité de créer des types de données spécifiques nommés pour les besoins de l'utilisateur

```
<xsd:simpleType name="Temperature">  
  <xsd:restriction base="xsd:double">  
    <xsd:minExclusive value="-273.15"/>  
  </xsd:restriction>  
</xsd:simpleType>
```

LES SCHÉMAS XML

- Les schémas XML apportent les bénéfices suivants:
 - La notion d'héritage: les éléments peuvent hériter du contenu et des attributs d'un autre élément.

```
<xs:complexType name="AddressType">
  <xs:sequence>
    <xs:element name="Line1" type="xs:string" />
    <xs:element name="Line2" type="xs:string" />
  </xs:sequence>
</xs:complexType>
<xs:complexType name="UKAddressType">
  <xs:complexContent>
    <xs:extension base="AddressType">
      <xs:sequence>
        <xs:element name="County" type="xs:string" />
        <xs:element name="Postcode" type="xs:string" />
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

LES SCHÉMAS XML

➤ Exemple de schéma xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="shiporder">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="orderperson" type="xs:string"/>
        <xs:element name="shipto">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="name" type="xs:string"/>
              <xs:element name="address" type="xs:string"/>
              <xs:element name="city" type="xs:string"/>
              <xs:element name="country" type="xs:string"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
        <xs:element name="item" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="title" type="xs:string"/>
              <xs:element name="note" type="xs:string" minOccurs="0"/>
              <xs:element name="quantity" type="xs:positiveInteger"/>
              <xs:element name="price" type="xs:decimal"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
      <xs:attribute name="orderid" type="xs:string" use="required"/>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

➤ Exemple de DTD pour le même model

```
<!ELEMENT shiporder( orderperson, item)>
<!ELEMENT orderperson(#PCDATA)>
<!ELEMENT item(#PCDATA)>
```

LES SCHÉMAS XML

- Les schémas XML permettent de représenter des métadonnées pour décrire des structures arbitrairement complexes
- Les parseurs basés sur les schémas XML sont disponibles et permettent de faciliter le travail des applications manipulant des documents XML
- L'accès à la description complète des structures de données favorise l'automatisation des échanges entre applications
- Par exemple, les schémas XML sont utilisés:
 - Dans les gestionnaires de bases de données relationnelles pour importer ou exporter les méta-données.

SERVICES WEB

- Les web services sont des composants qui offrent leur service à travers le web
- Ils sont accessibles par des protocoles standards
- Ils sont décrits à partir de XML
- Ils interagissent via des messages au format XML
- Ils sont localisables à partir de registres
- Ils sont complètement indépendants des plateformes et très faiblement couplés
- Ils introduisent un nouveau modèle de développement basé sur les architectures orientées service (SOA)
- Un service web est conçu pour résoudre un problème particulier
- Les services peuvent être combinés pour résoudre des problèmes complexes

SERVICES WEB: FONCTIONNEMENT

- Les tâches associées à la manipulation des services web sont:
 - Interroger un annuaire: qui fournit des services dont on ne connaît pas nécessairement la nature, le rôle ou le contenu
 - Négocier avec les fournisseurs de ces services pour connaître:
 - Nature exacte du service fourni
 - Qualité/coût/etc.
 - Interagir avec le service du fournisseur sélectionné pour:
 - Connaître les modalités d'interaction
 - Introduire le service dans ma chaîne de traitements
 - Eventuellement composer des services pour répondre à un problème complexe
 - Eventuellement publier mes propres services dans l'annuaire pour que d'autres puissent également les réutiliser

SERVICES WEB: LES ACTEURS

Les principaux acteurs dans la technologie des services web sont:

- Le client: c'est celui qui utilise / invoque le service
- Le fournisseur: c'est celui qui met à disposition le service web
- L'annuaire: celui qui détient les informations du service web

SERVICES WEB: LES ACTEURS

➤ Étape 1: définition, description du service

- On doit décrire d'un point de vue informatique ce que fait le service
- Cette définition est réalisé en WSDL au sein du fournisseur de service (le serveur qui expose le service)

➤ Étape 2: Publication du service:

- Une fois le service défini en termes de mise en œuvre, il peut être déclaré dans un annuaire afin de le rendre accessible aux clients: on parle de publication de service
- Cette publication est faite au sein d'un annuaire dédié : UDDI

➤ Étape 3: recherche du service

- Le client se connecte sur un annuaire UDDI pour rechercher un service

SERVICES WEB: LES ACTEURS

➤ Étape 4: enregistrement au service web

- Une fois l service trouvé par le client, ce dernier doit s'enregistrer auprès du fournisseur. Il indique ainsi au fournisseur son intention d'utiliser le service conformément aux conditions décrites dans la publication.

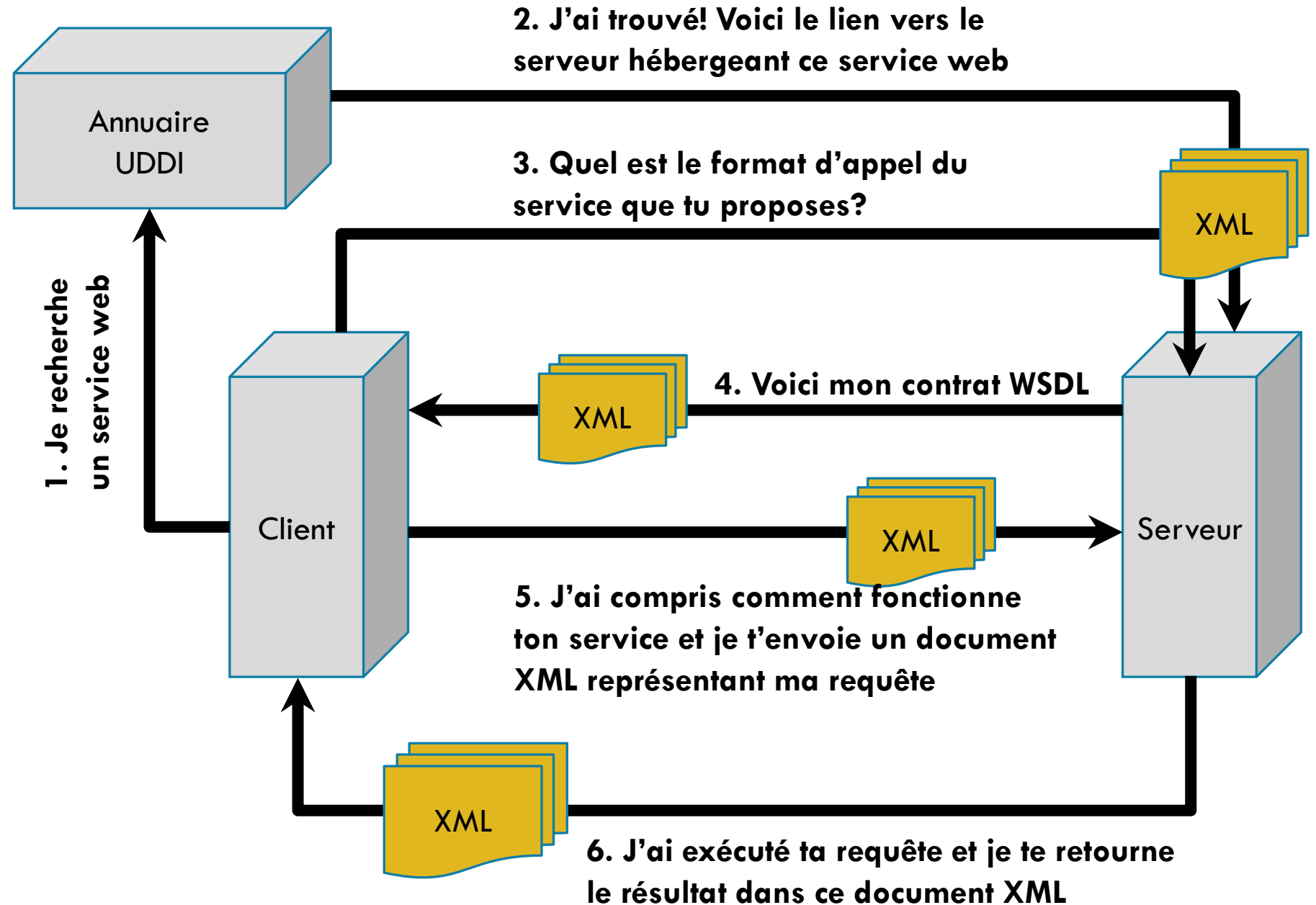
➤ Étape 5: Mise en œuvre du service

- Le client peut invoquer le service suivant les conditions décrites dans la publication

➤ Étape 6: Composition

- C'est la possibilité de combiner plusieurs services, un service pouvant devenir client d'un autre service

ACTEURS



SERVICES WEB: LES TECHNOLOGIES

Les protocoles

Protocoles	Description
WSDL	Document XML décrivant le service. Il indique son emplacement et spécifie les différentes méthodes du service
SOAP	Protocole de communication entre applications basé sur XML. Il permet l'échange de données structurées entre applications.
HTTP	Protocole de la couche réseau: HyperText Transport Protocol

SERVICES WEB: LES TECHNOLOGIES

- Afin de permettre la publication et la recherche des services web, deux couches supplémentaires sont nécessaires.
 - Une couche UDDI pour la publication de services
 - Une couche UDDI pour la découverte de services
- Les services sont publiés via leur document WSDL auprès d'un annuaire UDDI
- Une application cliente peut découvrir et accéder aux services lors de son exécution via un annuaire UDDI.



LE PROTOCOLE SOAP

LE PROTOCOLE SOAP

➤ Rôle

- Assure les appels de procédures à distance au dessus d'un protocole de transport: HTTP, SMTP, ...

➤ Fonctionnement côté client

- Ouverture d'une connexion HTTP
- Requête SOAP sous forme d'un document XML décrivant
 - Une méthode à invoquer sur une machine distante
 - Les paramètres de la méthode

➤ Fonctionnement côté serveur

- Récupère la requête
- Exécute la méthode invoquée avec les paramètres
- Renvoie une réponse SOAP sous la forme d'un document XML au client

LE PROTOCOLE SOAP

- SOAP signifie **Simple Object Access Protocol**
- SOAP est un protocole de communication entre applications
- SOAP est un format d'envoi et de réception de messages
- SOAP est indépendant de toute plateforme
- SOAP est basé sur XML
- SOAP est une recommandation du W3C

LE PROTOCOLE SOAP

- Les messages envoyés par le protocole SOAP sont encapsulés dans une enveloppe SOAP
- L'enveloppe SOAP est une grammaire prédéfinie
- La grammaire du message dépend de l'application car elle décrit les méthodes et les paramètres applicatifs
- SOAP est utilisables avec différents types de protocoles:
 - Protocoles synchrones: HTTP
 - Appels bloquants
 - Pas de garantie de résultat (timeout)
 - Protocoles asynchrones: SMTP, JMS, ...
 - Appels non bloquants
 - garantie de service (exactly once)

LE PROTOCOLE SOAP

- Le format SOAP est constitué de quatre parties
 - Une enveloppe permettant d'identifier le document XML comme étant un message SOAP
 - Un prologue (header) contenant des informations d'en-tête généralement utilisées pour l'authentification et la gestion de session
 - Un corps (Body) contenant les informations sur les appels et les réponses
 - Un élément Fault contenant les messages d'erreurs et le statuts
- Ces quatre éléments sont tous définis dans le namespace par défaut de SOAP: <http://www.w3.org/2003/05/soap-envelope/> et le namespace d'encodage <http://www.w3.org/2003/05/soap-encoding>

LE PROTOCOLE SOAP

- Un message SOAP doit être obligatoirement un document XML
- Un message SOAP doit obligatoirement être contenu dans une enveloppe SOAP et utiliser les namespaces d'encodage SOAP
- Un message SOAP ne doit contenir aucune référence à une DTD
- Un message SOAP ne doit contenir aucune instruction de traitement XML

LE PROTOCOLE SOAP

```
<?xml version="1.0"?>
```

```
<soap:Envelope  
  xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"  
  soap:encodingStyle="http://www.w3.org/2003/05/soap-  
  encoding">
```

```
  <soap:Header>
```

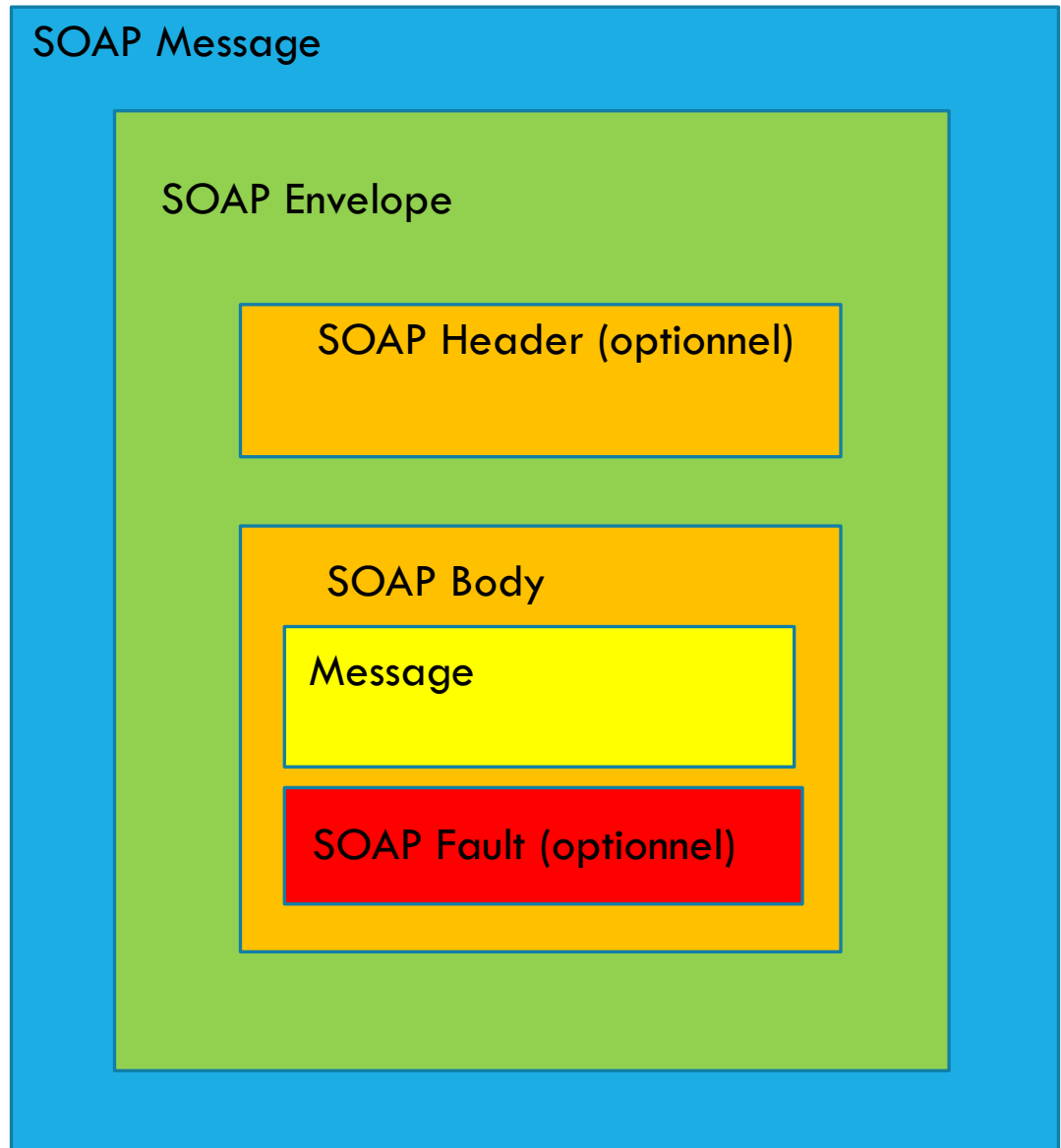
```
    ...  
  </soap:Header>
```

```
  <soap:Body>
```

```
    ...  
    <soap:Fault>
```

```
      ...  
    </soap:Fault>  
  </soap:Body>
```

```
</soap:Envelope>
```



LE PROTOCOLE SOAP

L'exemple de document SOAP ci-dessous est une requête demandant le prix des pommes. « GetPrice » est a méthode à invoquer et l'élément « Item » est le paramètre à passer à cette méthode. Les éléments « GetPrice et Item sont donc des éléments spécifiques à l'application.

```
<?xml version="1.0"?>
```

```
<soap:Envelope  
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"  
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">
```

```
<soap:Body>  
  <m:GetPrice xmlns:m="https://www.w3schools.com/prices">  
    <m:Item>Apples</m:Item>  
  </m:GetPrice>  
</soap:Body>
```

```
</soap:Envelope>
```

méthode java à exécuter par la
requête ci-contre:

```
public double GetPrice(Item item)
```

LE PROTOCOLE SOAP

La réponse à la requête précédente pourrait être un document SOAP comme le suivant indiquant un prix de 1.90 pour les pommes.

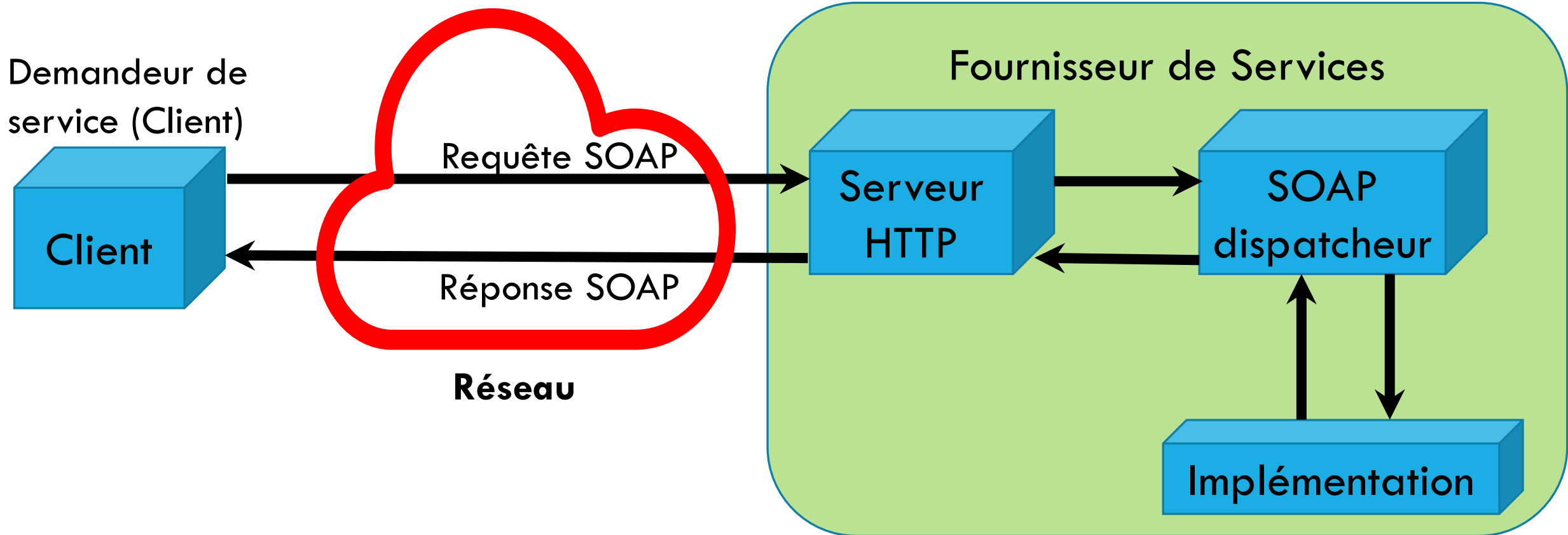
```
<?xml version="1.0"?>
```

```
<soap:Envelope  
  xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"  
  soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">
```

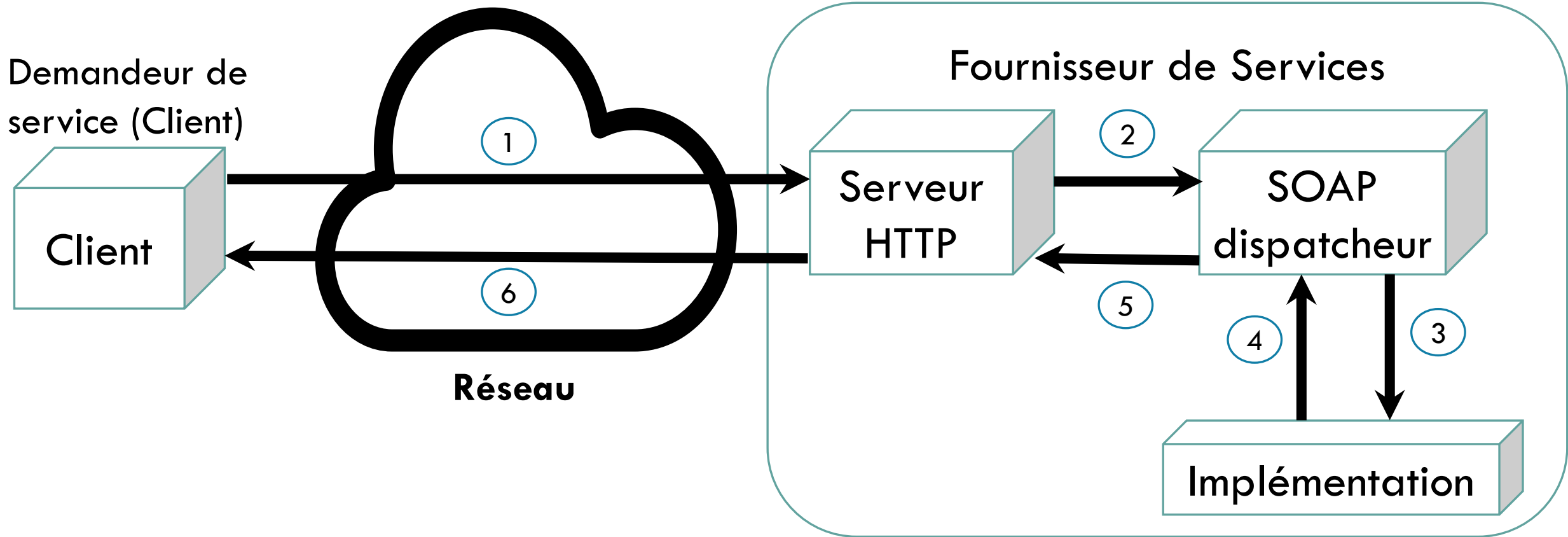
```
  <soap:Body>  
    <m:GetPriceResponse xmlns:m="https://www.w3schools.com/prices">  
      <m:Price>1.90</m:Price>  
    </m:GetPriceResponse>  
  </soap:Body>
```

```
</soap:Envelope>
```


LE PROTOCOLE SOAP: VISION GLOBALE



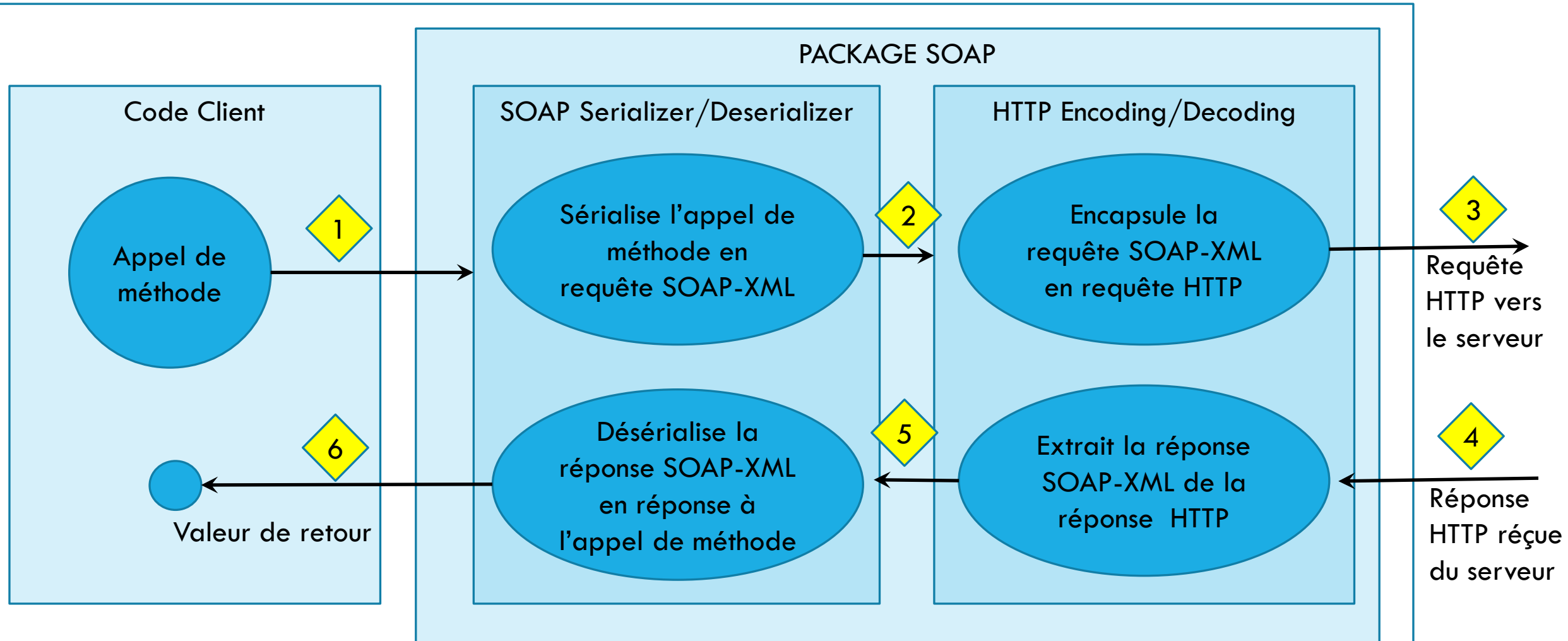
LE PROTOCOLE SOAP: VISION GLOBALE



LE PROTOCOLE SOAP: ARCHITECTURE CÔTÉ CLIENT

- Les messages envoyés au serveur par le client seront des requêtes SOAP-XML encapsulées dans des requêtes HTTP.
- De même les réponses retournées par le serveur sont des réponses HTTP qui renferment des réponses SOAP-XML
- En pratique côté client, on utilisera un package SOAP dont on invoquera la méthode appropriée. C'est le package qui sérialisera l'invocation ainsi effectuée en requête SOAP puis l'encapsulera dans une requête HTTP avant de l'envoyer à l'URL du serveur
- De même au retour de la réponse du serveur, c'est ce package qui s'occupera de décoder le message HTTP reçu, d'en extraire la réponse SOAP avant de désérialiser ce dernier afin d'obtenir la valeur de retour. Cette valeur de retour est ensuite passée comme valeur de retour à l'invocation originale de la méthode par le client.

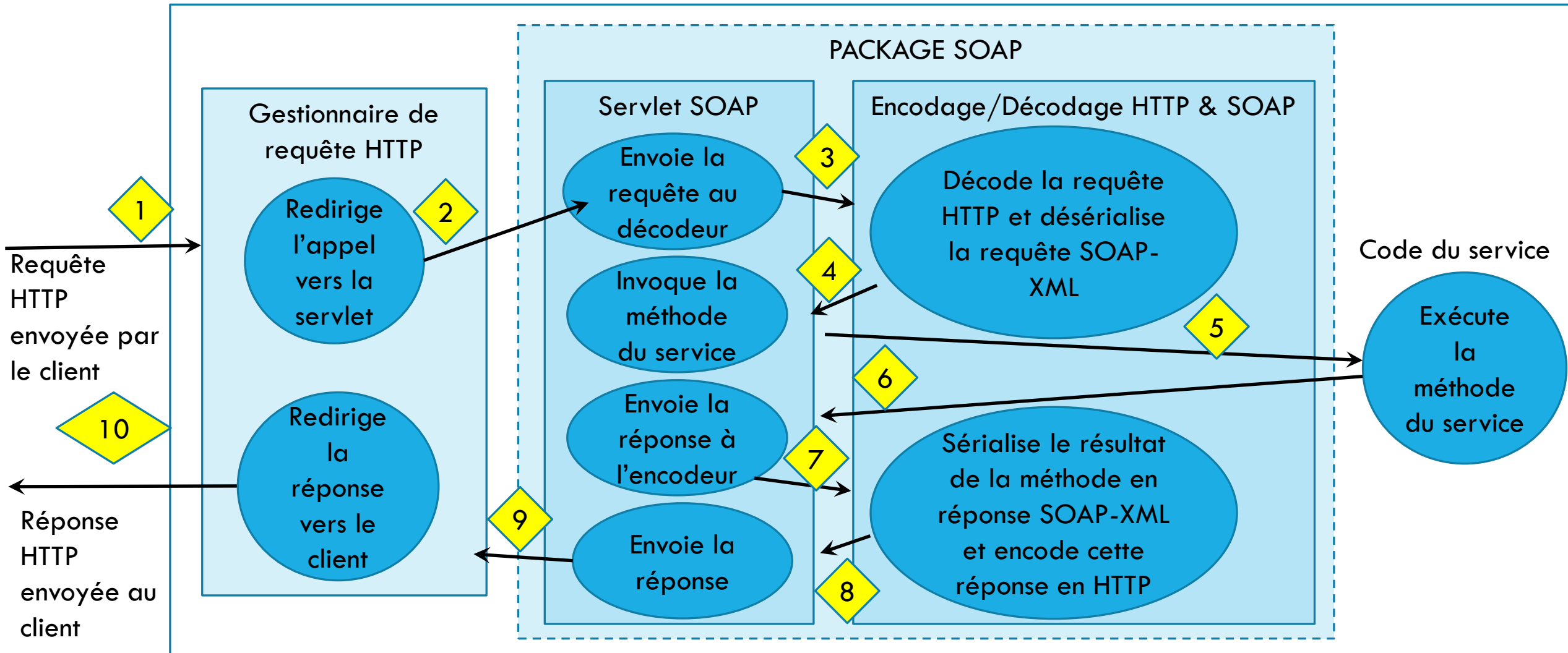
LE PROTOCOLE SOAP: ARCHITECTURE CÔTÉ CLIENT



LE PROTOCOLE SOAP: ARCHITECTURE CÔTÉ SERVEUR

- Côté serveur, on s'appuie sur un « process » listener, c'est-à-dire un processus qui écoute de façon active l'arrivée de requêtes clientes.
- Le listener est souvent implémenté par le moyen d'une servlet qui s'exécute sur un serveur d'application.
- Le travail de la servlet SOAP consiste à extraire le message SOAP-XML de la requête HTTP reçue de la part du client, de le désérialiser en identifiant la méthode et les paramètres fournis, puis à invoquer la méthode du service. Il récupère alors le résultat de l'invocation, le sérialise dans un format SOAP-XML, encode le tout dans une réponse HTTP qu'il retourne au client.
- Tout comme côté client, la servlet SOAP s'appuie sur un package SOAP pour sérialiser / désérialiser, encoder / décoder les messages.

LE PROTOCOLE SOAP: ARCHITECTURE CÔTÉ SERVEUR





LE LANGAGE WSDL

LE LANGAGE WSDL

- WSDL signifie Web Services Description Language
- Le document WSDL est une interface qui cache le détail de l'implémentation d'un service web et permet de le rendre indépendant :
 - de la plateforme utilisée
 - du langage de programmation utilisé
- Le fichier WSDL est au format XML et regroupe l'ensemble des informations permettant de communiquer avec le service web:
 - Les méthodes
 - Les paramètres des méthodes ainsi que les valeurs que celles-ci retournent
 - Le protocole de transport utilisé
 - La localisation du service web
- En pratique le développeur ne décrit pas lui-même le contenu du document WSDL. Les outils de développement permettent de le générer

LE LANGAGE WSDL

- Il existe deux types de documents WSDL
 - le document WSDL décrivant l'interface du service
 - Le document WSDL décrivant l'implémentation du service
- Ces deux documents sont indispensables au déploiement des services web
- Par exemple la publication et la recherche de services dans l'annuaire UDDI se font via ces deux types de documents WSDL

LE LANGAGE WSDL

Un document WSDL décrit un service Web. Il spécifie l'emplacement du service et les méthodes du service, en utilisant les principaux éléments suivants:

- **Les types:** un type décrit la structure de données utilisée par le service et transmise dans un message
- **Les messages:** un message décrit l'ensemble des données transmises au cours de l'opération. Il peut être une requête ou une réponse.
- **Les types de port:** un type de port décrit les opérations qui peuvent être effectuées et les messages impliqués.
- **Les liaisons:** la liaison décrit la manière dont un ensemble d'opérations abstraites, c'est-à-dire un type de port, est lié à un port selon un protocole réel

LE LANGAGE WSDL

<definitions>

<types>

definition des types de données...

</types>

<message>

definition des données échangées....

</message>

<portType>

ensemble d'opérations.....

</portType>

<binding>

protocole et spécification du format de données....

</binding>

</definitions>

LE LANGAGE WSDL

Un document WSDL comprend également deux autres parties:

- **Les ports:** un port spécifie une adresse qui est associée à une « liaison » définissant un simple point de terminaison
- **Le service:** un service est décrit comme un ensemble de points de terminaisons de réseau appelés « ports »
- Voici un exemple complet de [document WSDL](#)



L'ANNUAIRE UDDI

L'ANNUAIRE UDDI

➤ Rôle

- Spécification pour la définition d'un service de registre

➤ Fournisseur

- Déclaration du fournisseur
- Enregistrement des services web disponibles du fournisseur

➤ Client

- Requête de recherche de services web via SOAP
- Mise en relation avec le service web d'un fournisseur

L'ANNUAIRE UDDI

- UDDI signifie: Universal Description, Discovery and Integration
- UDDI est né sous l'impulsion de Microsoft et IBM suivis par de nombreux autres industriels dont Oracle, Sun, HP, Intel.
- Le standard UDDI a été mis au point pour faciliter la collaboration entre partenaires dans le cadre d'échanges commerciaux (développement d'échanges B2B)
- UDDI est un annuaire contenant des informations techniques et administratives sur les entreprises et les services que celles-ci publient
- UDDI permet d'effectuer différentes recherches
 - Recherche par catégories d'entreprises en utilisant des taxonomies ou des identifications d'entreprises
 - Recherche d'un service particulier
- UDDI repose sur le protocole SOAP et de ce fait les requêtes et réponses sur l'annuaire sont des messages SOAP.

L'ANNUAIRE UDDI

L'annuaire UDDI comprend plusieurs catégories de données:

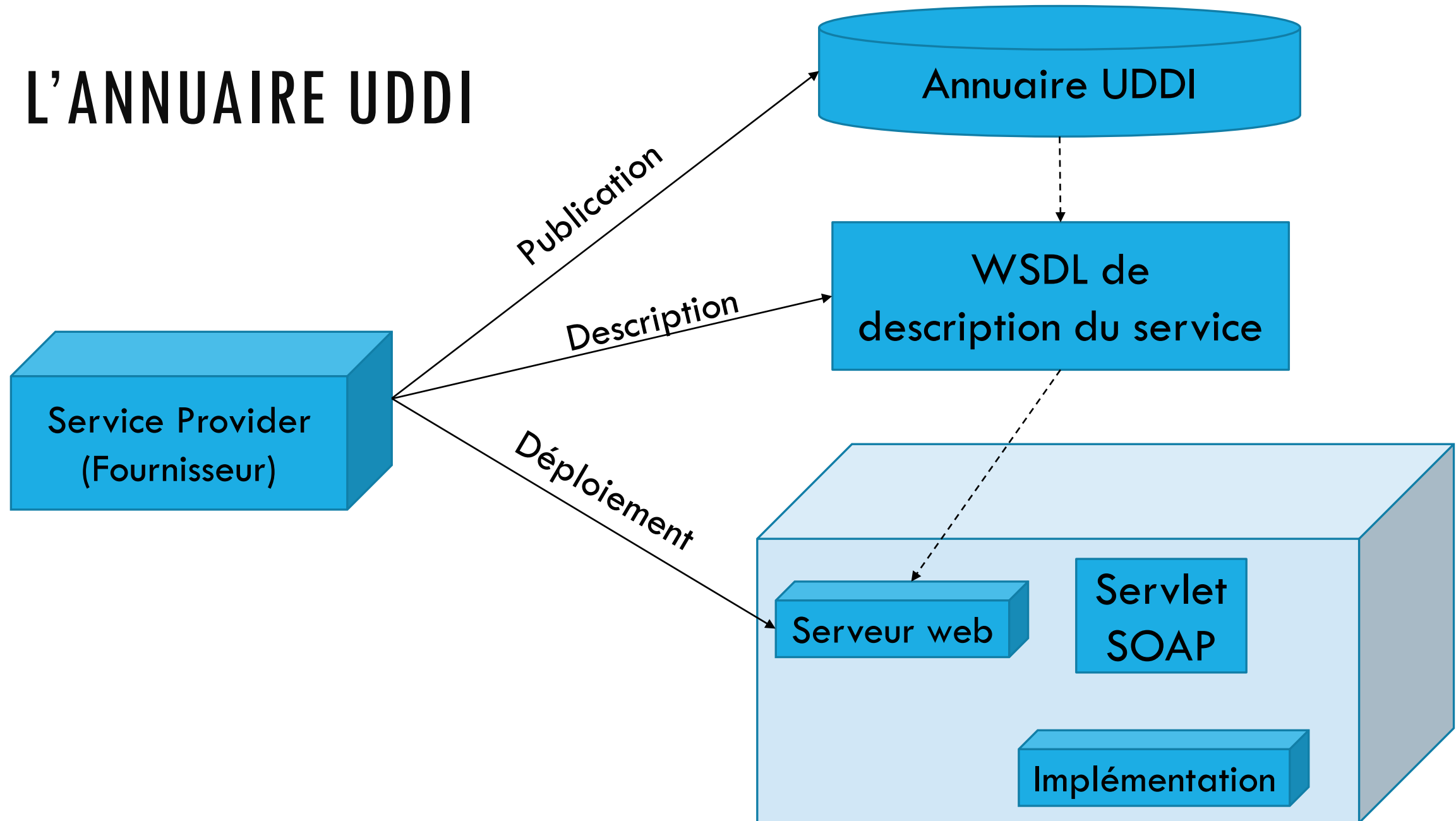
- Pages blanches: elles recensent les entreprises et contiennent les informations comme les suivantes:
 - Le nom de 'entreprise
 - Les coordonnées de l'entreprise
 - Des descriptions accessibles aux clients
 - Des identifiants permettant de la retrouver par recherche
- Pages jaunes: Elles contiennent la description des services déployés par l'entreprise. Les services sont classés par catégorie.
- Pages bleues: Elles fournissent des informations techniques détaillées sur les services

L'ANNUAIRE UDDI: PUBLICATION DE SERVICES

Pour une publication dans un annuaire UDDI, il existe plusieurs possibilités, suivant la visibilité du service voulue par le fournisseur:

- **Nœud interne UDDI:** on l'utilise pour une application interne à l'entreprise. Ce type de nœud est protégé par un firewall et utilisé en intranet.
- **Nœud catalogue du partenaire avec UDDI:** les services sont publiés sur un nœud catalogue d'un partenaire choisi avec les autorisations d'accès adéquates
- **Nœud portail UDDI** pour une application extranet. Les services sont publiés pour que les partenaires externes puissent les utiliser
- **Nœud place de marché UDDI** pour une relation inter-entreprises. Les services sont largement accessibles pour un partage des ressources entre systèmes d'information.

L'ANNUAIRE UDDI



L'ANNUAIRE UDDI

REGISTRE

